



NPU 单核架构初版设计介绍

[!info] 文档用途 本文是面向 PPT 的架构摘要，集中说明单核第一版的设计目标、指令、硬件模块、重点算子和软件分层。详细字段、模块接口与周期时序以《NPU 指令与硬件架构设计 Spec》为准。

1. 设计目标

- 面向 Transformer、RNN、LSTM、GRU 和回归模型的整数推理，兼顾自然语言处理与时序数据分析。
- 第一版采用单核结构：一个 Matrix Engine、一个 Integer Vector Engine、一个 Complex Math Engine 和一个 DMA / Layout Engine。
- Generic Core 是 NPU 外部的主控 CPU，不计入 NPU 单核硬件。主控 CPU 运行模型调用代码与 C 驱动，以 AXI Master 身份主动访问系统总线。
- NPU 对系统总线提供 64-bit AXI Slave 接口。该接口包含命令提交、控制寄存器和 L1BUF 外部访问窗口；CFE 只接收并解析命令，不发起 AXI 访问。
- 模型张量支持 INT4、INT8、INT16 和 INT32。INT4/INT8 用于输入、权重和紧凑输出；INT16 用于回归模型或需要更细整数间隔的输入、权重、中间张量和输出；INT32 用于矩阵累加、bias 和较大数值范围的中间结果。
- 外部系统总线和模块间数据接口均以 64 bit 为一个 beat；命令通过固定地址的 CMD FIFO 数据端口写入，一条 128-bit 命令按低、高两个 beat 连续发送。
- 事件表示任务依赖；TaskScheduler 仅在等待事件满足且资源可用时发射任务。Matrix 的乘累加阶段与部分和阶段只在读写地址范围安全时并行，使数据搬运、矩阵乘、部分和处理和复杂函数按数据依赖推进。
- 复杂函数采用 INT → FP32 → INT：FP32 只在 Complex Math Engine 内部出现，不作为软件可见的模型张量格式。
- 第一版没有卷积硬件单元，也没有卷积指令。仅处理 NHWC、groups=1，且 stride、dilation 与 padding 可由当前 DMA Descriptor 表达的 Conv2D。编译器将其拆分为 padding / im2col / DMA FILL / COPY_ND / GEMM / bias / 输出整理，复用 DMA 和 Matrix Engine；数据格式、分组、属性或所需 L1BUF 工作区不符合要求时，在编译阶段报告错误。

[!note] P0 与 P1 P0 是单核第一版必须实现并验证的功能；P1 是已经预留编码、可在后续版本启用的功能。软件必须先读取设备功能寄存器，不能向未启用的 P1 功能发出任务。

1.1 C 语言参考模型配置

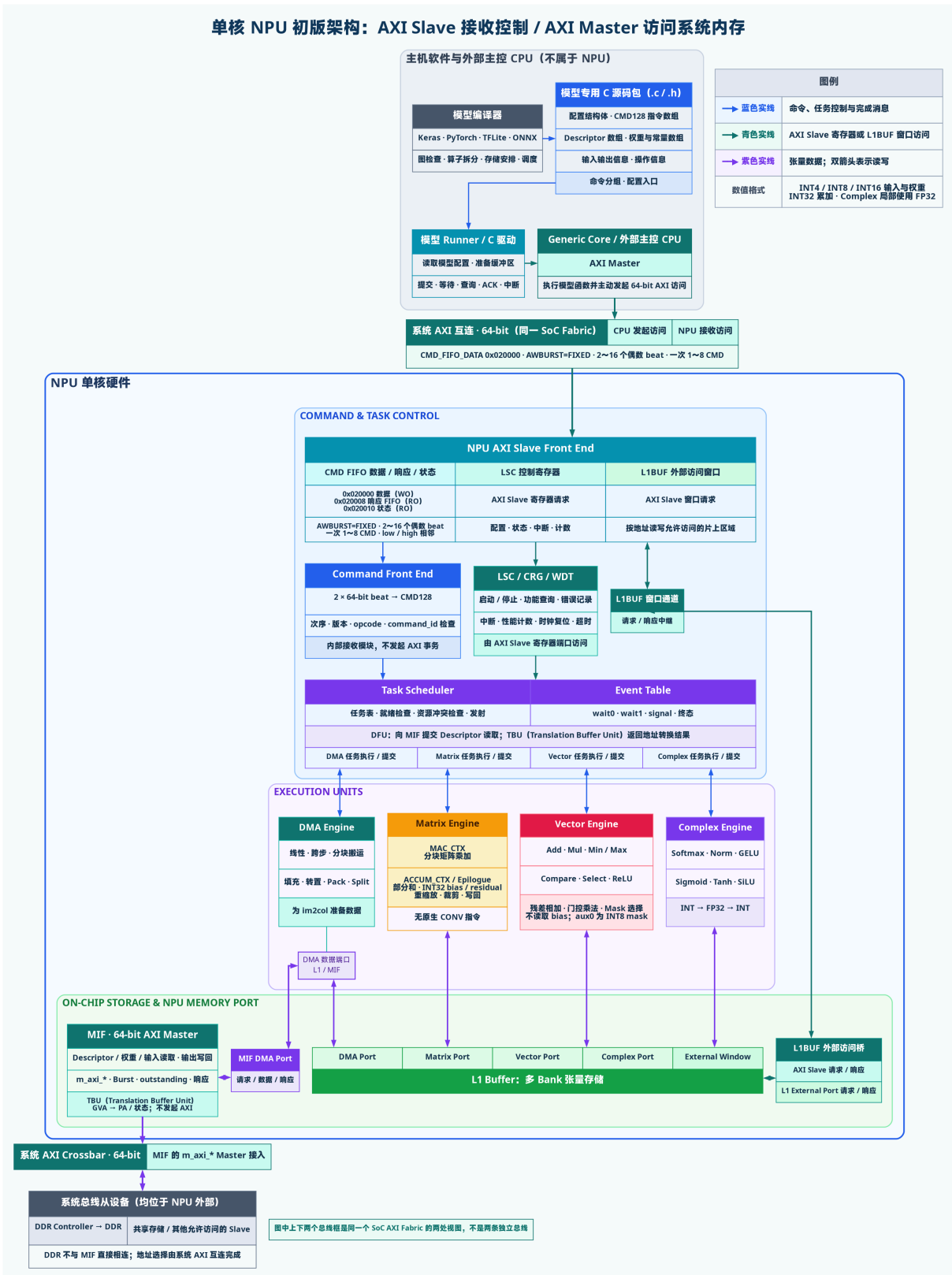
项目	参考值	项目	参考值
NPU Core	1	任务表项	32（终态、未 ACK 的任务仍占用）
L1BUF	1 MiB、16 bank	命令 ingress 缓冲	16 beat（最多 8 条 CMD128）
Matrix tile	$MT \times KT \times NT = 8 \times 16 \times 8$	Vector lane	8
Complex FP32 lane	4	DMA 未完成请求上限	16
外部地址	48-bit GVA → 40-bit PA	事件表	255 项

GVA（Global Virtual Address）由 TBU（Translation Buffer Unit，地址转换缓冲单元）转换为 PA（Physical Address）。命令 ingress 缓冲以 16 个 64-bit beat 暂存一个合法 burst；Command Front End（CFE）内部命令 FIFO 另有 8 个完整 CMD128 表项。

这些物理数值不写入指令编码。RTL 可以调整 L1BUF 容量、bank 数、tile 和 lane 数，并通过只读功能寄存器报告；编译器读取实际配置后选择分块与任务规模。

2. 单核总体结构

单核 NPU 初版架构：AXI Slave 接收控制 / AXI Master 访问系统内存



2.1 访问方向与模块职责

图中用不同颜色区分命令控制、计算、片上存储和外部访问。主要访问方向如下：

命令与控制：

外部 Generic Core (AXI Master)

→ 系统 AXI 总线 → NPU AXI Slave → CFE

→ TS 保存任务 → DFU 读取并检查 Descriptor

→ TS 等待 Event → 执行单元

全局内存：

DFU / DMA —— GVA 请求 ——> MIF

MIF —— 地址转换请求 ——> TBU

TBU —— PA 或地址错误 ——> MIF

MIF —— 'm_axi_*': 64-bit AXI Master ——> 系统 AXI 互连

系统 AXI 互连 —— 按 PA 选择目标 ——> DDR Controller / 共享存储 / 其他允许访问的 Slave

DDR Controller —— 内存接口 ——> DDR

片上数据：

┌ Matrix (MAC + Epilogue; 按需读取 INT32 bias)

DMA ↔ L1BUF ── Vector (逐元素整数运算; 不读取 bias)

└ Complex (激活、Softmax、Norm)

- Command Front End (CFE) 接收来自 NPU AXI Slave 的两个命令 beat，检查次序、版本、操作码和命令编号，再把完整命令送入命令 FIFO。CFE 不具有 AXI Master 端口。
- TaskScheduler (TS) 先把 CFE 送来的命令保存到任务表；Descriptor Fetch Unit (DFU) 随后读取描述符并检查地址、shape、stride、dtype 与保留位。Descriptor 检查通过后，TS 再检查 Event 和执行单元状态，满足条件时发射任务。
- 四个执行单元通过 L1BUF 交换张量；Matrix、Vector 和 Complex 不直接访问 DDR。
- L1BUF 的计算端口供 DMA、Matrix、Vector 和 Complex 使用；LSC 必须显式开启外部访问窗口，外部 CPU 才可通过 AXI Slave 地址窗口读写允许访问的片上区域。硬件仲裁外部与计算端口的读写请求；软件在读写仍可能被任务访问的 L1BUF 地址前，必须等待相应事件或任务栅栏完成。
- Memory Interface (MIF) 接收 NPU 内部的存储访问请求，并通过 64-bit m_axi_* 接入系统 AXI 互连。MIF 在 TBU 返回有效物理地址后才发出 AXI 读写请求；DDR Controller 与其他系统目标由 SoC 的 AXI 互连按物理地址选择。
- Translation Buffer Unit (TBU，地址转换缓冲单元) 处理 MIF 提交的地址转换请求，返回物理地址、权限错误或页表错误；发生地址转换错误时，MIF 不发出对应的 AXI 请求。
- LSC 提供可由 AXI Slave 访问的寄存器、中断、性能计数和启动停止控制；CRG 与 WDT 分别负责时钟复位和超时监视。RTL 复位信号统一使用低有效 reset_n。

[!note] MIF 与 TBU 的分工 MIF (Memory Interface)

有一组 64-bit AXI Master 接口 m_axi_*。

TBU (Translation Buffer Unit，地址转换缓冲单元) 向 MIF 返回虚拟地址到物理地址的转换结果及状态；MIF 随后把 PA 放到这组 AXI Master 接口上。系统 AXI 互连根据 PA 选择 DDR Controller、共享存储或其他允许访问的 Slave，因此 MIF 不需要识别 DDR 与其他系统目标，也不具有 DDR 专用端口。TBU 本身不发出 AXI 事务。第一版的常规地址为 48-bit GVA，转换后的地址为 40-bit PA。

- Generic Core 位于 NPU 框外。示例 runner 读取模型专用 C 源码包中的配置、描述符、权重和 CMD128，并通过 C 驱动准备缓冲区、提交命令；CPU 以 AXI Master 身份主动发起写事务。
- NPU AXI Slave 把地址访问分送到固定地址 CMD FIFO 数据端口、LSC 控制寄存器或 L1BUF 外部访问桥。CPU 使用 AWBURST=FIXED 向 CMD FIFO 连续写入 2~16 个 64-bit beat；每条 CMD128 固定采用 low beat、high beat 的次序，同一 burst 可以依次携带 1~8 条完整命令。外部访问桥把 AXI Slave 的单项请求与 L1BUF 外部端口的读写通道相互转换。

[!important] 主设备与从设备的方向 Generic Core 是系统总线主设备；NPU 命令端口、控制寄存器端口和 L1BUF 外部窗口都是系统总线从设备。MIF 也具有主设备能力，但只有一组 m_axi_*，并与 Generic Core 一样接入 SoC 系统 AXI 互连。DDR Controller 是该互连上的从设备之一，不与 MIF 直接相连。TBU 只提供地址转换服务，不是 AXI Master。CFE 是 NPU 内部命令接收模块，不是总线主设备。

3. 为什么采用 CMD128

一条任务需要同时携带描述符地址、命令编号、执行单元、完整操作码、两个等待事件、一个完成事件以及任务选项。若压缩成 64 bit，就需要额外写共享配置寄存器，增加寄存器访问、设备写屏障和多提交者互斥处理。CMD128 把一次提交所需信息放在同一条命令中，同时保持物理接口为 64 bit。

3.1 低 64 bit

CMD bit	字段	说明
[47:0]	desc_addr	48-bit 描述符字节地址，要求 64B 对齐
[59:48]	command_id	12-bit 软件命令编号
[63:60]	engine	Control、DMA、Matrix、Vector 或 Complex

3.2 高 64 bit

CMD bit	字段	说明
[71:64]	opcode	完整 8-bit 操作码
[83:72]	header_flags	任务选项与超时类别；具体字段以 Spec 为准
[95:84]	wait_event_0	第一个等待事件
[107:96]	wait_event_1	第二个等待事件
[119:108]	signal_event	任务进入终态时更新的事件
[127:120]	header_version	V1.1 写 0x01

外部 CPU 不发送 **first/last** 标记。驱动把每条 CMD128 排成相邻的低、高 word，并向固定地址 **CMD_FIFO_DATA** 发出 2~16 beat 的偶数长度 FIXED burst，一次可提交 1~8 条命令。Front End 先暂存完整 burst；只有 beat 数、**WSTRB** 和 **WLAST** 全部正确时，才把整个 burst 放入 ingress FIFO，否则 整体拒绝。进入 NPU 内部后，Front End 再根据低、高 word 的位置产生 **first/last** 信号。CMD128 只携带提交与调度信息；Descriptor 承载数据地址、shape、stride、数据格式和执行单元专有参数。

4. Event 如何表达任务依赖

Event（事件）不是张量数据，也不是 CPU 中断。它是 TaskScheduler 的 Event Table 中一项很小的状态记录，用来回答一个问题：“生产者任务是否已经把消费者需要的数据准备好？”状态来自携带 `signal_event` 的任务终态、`EVENT_SIGNAL` 的显式成功通知，或 `EVENT_JOIN` 按 `join_mode` 写入的合并结果。张量仍保存在 L1BUF 或 DDR 中，数据地址仍由 Descriptor 给出；Event 只记录先后关系和完成结果。

一个事件引用占 12 bit：

$$\text{event_ref} = (\text{generation} \ll 8) | \text{event_id}.$$

字段	位数	含义
event_id	8	Event Table 项号；0~254 有效，255 保留
generation	4	同一 Event ID 的使用代次，取值 0~15
0xFFF	12	特殊值，表示该 CMD 不等待或不产生事件

例如 `event_id=0x25`、`generation=3` 时，事件引用为 `0x325`。以后重新使用 `event_id=0x25` 时，可以把 `generation` 增加到 4，得到 `0x425`。这样，即使上一代任务的完成消息很晚才到达，也不会被误认为新一代任务已经完成。

4.1 signal、wait 与失败传播

每条 CMD128 最多携带两个输入依赖 `wait_event_0/1` 和一个输出通知 `signal_event`：

- 任务被接收时，TS 为非 `0xFFFF` 的 `signal_event` 保留对应表项，并记录生产者 `command_id`。
- 任务的两个 `wait` 字段均为 `0xFFFF` 时，它不依赖其他任务；资源可用且 Descriptor 检查通过后即可发射。
- 非空 `wait` 必须同时满足 Event ID 与 `generation`。所有等待事件都为 `SUCCESS` 后，任务才进入可发射状态。
- 任一等待事件为 `ERROR` 时，消费者不读取可能无效的数据，也不进入执行单元，而是以 `DEPENDENCY_FAILED` 结束；该消费者的非空 `signal_event` 也改为 `ERROR`，使失败状态继续传给后续任务。
- 生产者只有在最后一次数据写入已经收到完成响应后，才把自己的 `signal_event` 改为 `SUCCESS`；生产者失败时则改为 `ERROR` 并保存失败状态。

事件表项的典型生命周期是：

FREE

- RESERVED（生产者 CMD 已被接收并保留该表项）
- SUCCESS 或 ERROR（生产者进入终态）
- EVENT_REARM（确认没有等待者）
- `generation` 按 4 bit 加 1，再回到 FREE

`generation` 的加法按模 16 处理，即 15 的下一代是 0。软件不得仅凭数值回绕就提前复用表项；必须先确认旧代次已进入终态、所有等待者均已处理，并完成 `EVENT_REARM`。这样，正在队列中的旧事件引用不会与新生产者混淆。

[!note] 为什么 Event 有利于并行调度 CPU 可以一次提交多条任务，不必按“提交一条、等待一条”的方式串行控制。TS 只暂停真正依赖前序结果的任务；没有数据依赖的 DMA、Matrix、Vector 和 Complex 任务仍可在各自单元上同时推进。Event 因而既保证读取时机正确，又保留了不同执行单元之间的并行机会。

4.2 例子：Matrix 完成后再启动 Vector

假设 Matrix 任务 `command_id=0x121` 把 GEMM 结果写到 L1BUF，Vector 任务 `command_id=0x122` 随后对该结果执行 ReLU。编译器选择事件 `0x325`：

任务	wait_event_0	signal_event	作用
Matrix / GEMM	0xFFFF	0x325	产生矩阵结果
Vector / VRELU_I	0x325	0x026	读取矩阵结果并产生下一阶段结果

两条 CMD 可以在同一个 FIXED burst 中提交。TS 接收后可立即安排 Matrix；Vector 已经在任务表中，但在 `0x325` 进入 `SUCCESS` 前不会读取 L1BUF。Matrix 最后一项写请求完成后，TS 更新 `0x325`，Vector 随即获得发射资格。若 Matrix 返回地址错误或数值错误，`0x325` 进入 `ERROR`，Vector 直接以 `DEPENDENCY_FAILED` 结束。与此同时，一个不等待 `0x325` 的 DMA 任务仍可独立运行，这就是 Event 相比 CPU 逐条等待更能发挥硬件并行能力的原因。

5. 指令集合

单元	Opcode	指令	阶段	功能说明
Control	0x00	NOP	P0	不读写数据，仅完成常规任务状态处理。
Control	0x01	EVENT_SIGNAL	P0	将指令指定且已保留的事件显式置为 SUCCESS 。
Control	0x02	EVENT_REARM	P0	在旧代次已进入终态且没有等待者时，为同一 Event ID 启用下一代次。
Control	0x03	EVENT_JOIN	P0	按 join_mode 等待并合并两个输入事件的状态，写入输出事件。
Control	0x04	GLOBAL_FENCE	P0	等待本命令被接收前、由 Engine mask 选中的任务全部进入终态。
DMA	0x20	DMA_COPY_1D	P0	在源地址与目的地址之间连续复制一维元素序列。
DMA	0x21	DMA_COPY_ND	P0	按 1~5 维 shape 与 stride 复制数据。
DMA	0x22	DMA_FILL	P0	向目标数组填入指定常量。
DMA	0x23	DMA_TRANSPOSE_2D	P0	对二维矩阵执行实际转置。
DMA	0x24	DMA_PACK	P0	将多个等距源数据段拼接为连续目标数据。
DMA	0x25	DMA_SPLIT	P0	将连续源数据段写入多个等距目标位置。
Matrix	0x40	GEMM	P0	执行通用矩阵乘法。
Matrix	0x41	BMM	P0	按外层计数执行批量矩阵乘法。
Matrix	0x42	GEMM_ACCUM	P0	将多个 K tile 的乘积累加到 INT32 部分和。
Matrix	0x43	GEMM_ZERO	P0	清零 INT32 部分和存储区域。
Vector	0x60	VADD_I	P0	执行逐元素整数加法。
Vector	0x61	VSUB_I	P0	执行逐元素整数减法。
Vector	0x62	VMUL_I	P0	执行逐元素整数乘法。
Vector	0x63	VFMA_I	P0	执行逐元素整数乘加。
Vector	0x64	VMAX_I	P0	逐元素选择两个输入中的较大值。
Vector	0x65	VMIN_I	P0	逐元素选择两个输入中的较小值。
Vector	0x66	VCMP_I	P0	由 VECTOR_DESC.compare_mode 选择 EQ/NE/LT/LE/GT/GE，对 src0 与 src1 的同 dtype、同 scale INT4/8/16/32 元素逐项比较；结果写为 INT8 0/1 Boolean mask，供 VSEL_I 或 Complex 的 BOOLEAN mask 使用。
Vector	0x67	VSEL_I	P0	读取 aux0_addr 指向的独立 INT8 mask；0 选择 src0 ，非 0 选择 src1 。
Vector	0x68	VCLAMP_I	P0	将元素值限制在给定的最小值和最大值之间。
Vector	0x69	VRELU_I	P0	对整数输入执行 ReLU。
Complex	0x80	VACT_I	P0	执行 Sigmoid、Tanh、GELU 或 SiLU 激活。
Complex	0x81	VSOFTMAX_I	P0	执行逐行 Softmax。
Complex	0x82	VNORM_I	P0	执行 LayerNorm 或 RMSNorm。
Complex	0x84	VSTAT_I	P0	对每行执行求和、最大值或平方和统计。
Complex	0x86	VADD_RESCALE_I	P0	对 scale 不同的两个张量执行逐元素加法。
DMA	0x28	DMA_GATHER_ND	P1	按索引表读取数据块。
Complex	0x83	VROPE_I	P1	执行旋转位置编码（RoPE）。
Complex	0x85	VRECIP_I	P1	执行倒数或平方根倒数。

P1 指令在第一版中不启用，提交后返回 **ILLEGAL_OPCODE**；软件可使用已启用的 P0 指令组成相应功能。

每条 CMD 指向一个固定版本 Descriptor。公共 64B 前缀保存输入输出地址、bias 或 scale 地址、dtype、舍入方式和软件标记；DMA、Matrix、Vector、Complex 再使用各自的专有字段保存 shape、stride、tile 及函数参数。

6. 计算与存储模块

模块	主要职责
NPU AXI Slave Front End	接收外部 CPU 的 64-bit AXI 访问；固定地址 CMD FIFO 使用 FIXED burst，LSC 寄存器和 L1BUF 外部窗口保持各自的地址访问方式
Command Front End	从 CMD FIFO 取得相邻的低、高两个 64-bit beat，拼接并检查 CMD128；不发起 AXI 事务
DMA / Layout Engine	系统内存与 L1BUF 双向搬运；完成多维复制、填充、转置、拼接和拆分
Matrix Engine	执行 GEMM/BMM；支持 INT4、INT8、INT16 输入和权重，使用 INT32 累加，并可写出 INT4、INT8、INT16 或 INT32
Integer Vector Engine	执行逐元素整数运算，适合 residual、门控乘法、比较、选择和 ReLU；不读取 bias
Complex Math Engine	完成激活、Softmax、LayerNorm、RMSNorm、统计与不同 scale 的相加
L1BUF Controller	管理多 bank 片上 SRAM，为四个执行单元和外部访问窗口仲裁读写请求
L1BUF 外部访问桥	在 AXI Slave 窗口请求与 L1BUF 外部客户端读写通道之间转换请求和响应
MIF (Memory Interface)	接收 DFU 与 DMA 的全局内存请求；取得有效物理地址后，经 64-bit <code>m_axi_*</code> 接入系统 AXI 互连并处理响应
TBU (Translation Buffer Unit, 地址转换缓冲单元)	将 MIF 提交的虚拟地址转换为物理地址并返回状态；自身不发出 AXI 请求
CFE / TS / DFU	CFE 接收 CMD128；TS 管理任务与事件；DFU 读取并检查 Descriptor，再交由 TS 选择可发射任务
LSC / CRG / WDT	提供 AXI Slave 控制寄存器与中断、时钟复位、性能计数和任务超时保护

L1BUF 是计算单元之间的数据交换中心。编译器负责安排输入、权重、临时张量和输出的位置；硬件按 Descriptor 给出的地址和 stride 访问，不解析模型文件，也不推测 Kernel 的排列方式。外部 CPU 仅在 LSC 开启窗口且访问许可满足时使用 L1BUF 外部窗口；正常执行期间由 DMA 负责 DDR 与 L1BUF 之间的数据传送。软件在访问可能仍被任务读写的 L1BUF 地址前，必须等待相关事件或任务栅栏完成。

`bias` 仅由设置 `FINAL_OUTPUT=1` 和 `BIAS_ENABLE=1` 的 Matrix Epilogue 从 `aux0_addr` 读取，格式为 `[N] INT32`；Vector 的 `aux0_addr` 只用于独立 INT8 mask，不读取 `bias`。

6.1 一条任务如何流过硬件

示例 runner 读取生成的配置结构体、Descriptor、权重、输入输出信息和命令分组，再调用通用 C 驱动。若 SoC 的 CPU 与 DDR 不具备硬件缓存一致性，驱动在提交前清理 Descriptor、权重和输入的 CPU 缓存；CPU 读取输出前使对应缓存失效。随后驱动执行所需的设备内存屏障，并向固定地址 CMD FIFO 发起 `AWBURST=FIXED` 写事务，按 low word、high word 的次序连续发送命令 beat。CFE 接收并检查完整命令，TS 先把任务保存到任务表，DFU 再经 MIF 与 TBU 读取并检查 Descriptor。Descriptor 检查通过后，TS 等待所有非 `0xFF` 的前置 Event；任一前置 Event 以失败状态结束时，任务以 `DEPENDENCY_FAILED` 结束，全部成功且执行单元可接收时才发射任务。执行单元的最后一个写响应返回后，TS 才记录终态、更新完成事件并按命令选项产生中断。runner 通过驱动的查询、等待接口或中断取得结果，读取输出后调用 ACK 接口，释放任务表项和 `command_id`。

7. Matrix 两阶段并行

分块 GEMM 经常重复执行“新 tile 乘累加”和“旧 tile 部分和相加、后处理、写回”。若把二者放在同一不可分状态中，阵列会在旧结果写回期间闲置。第一版把 Matrix 任务拆成两个可独立推进的上下文，P0 各提供一个：

- **MAC_CTX**（乘累加上下文）：读取 A/B tile，执行乘累加，产生当前块结果。
- **ACCUM_CTX**（部分和累加与后处理上下文）：读取旧部分和，完成相加、bias、residual、整数重缩放、裁剪和写回。

P0 Matrix 支持 **INT4×INT4**、**INT8×INT8**、**INT8×INT4** 和 **INT16×INT16**；所有乘法结果累加到 INT32。**FINAL_OUTPUT=1** 时输出可为 INT4、INT8、INT16 或 INT32。dtype 编码 3 是 INT16；A/C 使用 pack 编码 5（**ROW_MAJOR_INT16**），B 使用 pack 编码 6（**B_KN_TILE_INT16**），每个元素按小端字节次序占 2B。

当旧任务进入 **ACCUM_CTX** 后，Matrix 乘法槽可以接收下一任务。若当前 **ACCUM_CTX.dst** 地址范围与候选 **MAC_CTX** 的 A 或 B 读取地址范围相交，调度器在当前任务的全部写响应返回前暂停候选 **MAC_CTX**，避免读到尚未完成写入的数据。当前 **ACCUM_CTX** 的只读地址与候选 A/B 相交时允许并行；候选任务的写后读与写后写关系在其进入 **ACCUM_CTX** 前检查。由于 P0 只有一个 **ACCUM_CTX**，两个 L1BUF 写阶段按提交次序执行。

任务的完成事件只能在最终写响应返回后更新，不能在乘法结束时提前更新。性能计数分别记录 MAC 活动周期、部分和活动周期、二者同时工作的周期以及地址等待周期，便于分析阵列利用率。

8. 面向 Transformer 的算子组合

1. Matrix 执行 Q、K、V 投影，或执行一次融合 QKV GEMM。
2. DMA 使用 COPY_ND、SPLIT、PACK 和 TRANSPOSE 整理 Head，并生成 K^T 。
3. Matrix 使用 BMM 计算 QK^T 。
4. Complex 完成 scale、mask 和逐行 Softmax。
5. Matrix 计算 $\text{Attention} \times V$ ，DMA 合并 Head，Matrix 完成输出投影。
6. Vector 完成 residual add；Complex 完成 LayerNorm 或 RMSNorm。
7. FFN 使用两次 GEMM，中间由 Complex 执行 GELU 或 SiLU。

注意力、FFN 和投影的大部分计算量由 Matrix Engine 承担；DMA 负责张量形状与存储次序调整；Vector 处理低成本逐元素运算；Complex 处理需要指数、平方根或除法的函数。

9. 面向 RNN、LSTM 与 GRU 的算子组合

- SimpleRNN: Matrix 分别计算输入项和隐藏状态项, Vector 相加, Complex 执行 Tanh; ReLU 版本可直接使用 Vector。
- LSTM: Matrix 一次生成输入门、遗忘门、候选值和输出门的仿射结果; Complex 执行 Sigmoid/Tanh; Vector 完成门值乘法、cell 更新与 hidden 更新。
- GRU: Matrix 生成 reset、update 和 candidate 的三组仿射结果; Complex 执行 Sigmoid/Tanh; Vector 完成门控乘法与状态组合。
- 时间步 $t + 1$ 必须等待 h_t , LSTM 还要等待 c_t 的最终写入完成。与此同时, DMA 可以把下一时间步输入和权重 tile 预取到另一片 L1BUF 区域。

门控模型中, 同一长度为 H 的 bias 会加到全部输入行: 第 j 个 bias 只加到每一行第 j 个隐藏 Feature, 而不是加到该行的其他位置。

10. Conv2D 使用 im2col + GEMM

第一版不增加卷积数据通路。仅接受 NHWC 输入、 $\text{groups}=1$ （每个输出通道均使用全部 C_{in} 输入通道），并要求 stride 、 dilation 与 padding 可由当前 DMA Descriptor 表达。编译器接收 $[N, H, W, C_{\text{in}}]$ 和 Kernel $[K_H, K_W, C_{\text{in}}, C_{\text{out}}]$ ，将每个输出位置对应的输入窗口展开成一行：

$$X_{\text{col}} \in \mathbb{Z}^{(NH_oW_o) \times (K_HK_WC_{\text{in}})}.$$

Kernel 在编译期整理为：

$$W_{\text{col}} \in \mathbb{Z}^{(K_HK_WC_{\text{in}}) \times C_{\text{out}}}.$$

随后执行：

$$Y_{\text{col}} = X_{\text{col}}W_{\text{col}} + b.$$

DMA FILL 先将 im2col 工作区填为零，再由 DMA COPY_ND 把各 Kernel 位置对应的有效输入片段写入该张量，Matrix GEMM 产生按 NHWC 线性次序保存的输出。输入与卷积核必须同为 INT4、INT8 或 INT16；INT16 情形使用 $\text{INT16} \times \text{INT16}$ GEMM。输出可为 INT4、INT8、INT16 或 INT32；可选 bias 由 Matrix Epilogue 读取，格式为 INT32 $[C_{\text{out}}]$ 。这样无需新增卷积 opcode，代价是额外的 L1BUF 容量和搬运次数；编译器需要按可用容量对输出位置分块。数据格式、分组、属性或所需工作区不符合要求时，编译阶段报告错误。

Complex Math Engine 对输入整数 q_x 执行：

$$x = (q_x - z_x)s_x \rightarrow r = f(x) \rightarrow q_y = \text{sat}(\text{round}(r/s_y + z_y)).$$

s_x, s_y 是输入和输出的实数缩放系数， z_x, z_y 是整数零点， f 可以是 Sigmoid、Tanh、GELU、SiLU、Exp 或平方根倒数，**round** 表示指定舍入方式，**sat** 表示限制到目标整数格式的可表示范围。Exp 与平方根倒数仅用于 Softmax 或 Norm 的内部计算步骤，不是可单独提交的 P0 指令。Softmax 与 Norm 也遵循相同的整数输入、内部 FP32 计算、整数输出次序。

FP32 scale、 ϵ 、函数系数和查找表属于只读元数据。FP32 中间值只保存在 Complex 内部寄存器或私有暂存区，不能写成模型输入、权重、中间张量或输出。

12. 模型编译结果与 C 驱动

```
Keras / PyTorch / TFLite / ONNX 模型
↓ npu_model_compiler.py
高层图检查与算子拆分
↓
低层 JSON IR (编译过程中的中间表示)
↓ npu_assembler.py
模型专用 C 源码包 (.c / .h)
├── 模型配置结构体
├── CMD128 指令数组
├── Descriptor 数组
├── 权重与常量数组
├── 输入输出与操作信息 (高层算子、执行单元、
opcode 与 command_id)
└── 命令分组 (一次 FIXED burst 提交的 1~8 条
CMD128)
↓ C 驱动
外部 Generic Core 经 C 驱动和系统 AXI 访问目标 NPU
硬件或 C 语言参考模型
```

- 上层编译器负责图检查、shape 推导、算子拆分、张量布局、L1/DDR 存储分配、Matrix-B tile 整理、事件依赖和任务调度；
MultiHeadAttention 与 **Conv2D** 在这一阶段拆成可以执行的步骤。
- 低层汇编器读取已经确定硬件字段的低层 JSON IR，编码 CMD128 与各类 Descriptor。它产生的字节内容随后写入 C 数组，不作为最终部署接口单独交给应用。
- 编译器的最终部署产物是模型专用 **.c** 与 **.h**：配置结构体说明输入输出、工作区、数组长度和所需设备功能；指令数组保存 CMD128；Descriptor 数组保存任务参数；权重数组保存离线整理后的整数权重和常量。
- 当前模型专用 C 源码包只提供配置结构体和静态数组，不生成模型初始化函数、模型运行函数或 CPU 辅助函数。操作信息记录高层算子对应的执行单元、opcode 与 **command_id**；命令分组记录一次提交的 CMD128 范围。示例 runner 读取这些信息后，调用通用 C 驱动完成缓冲区准备、CMD128 提交、等待、查询、中断处理和 ACK。
- C 驱动分为 device、command、runtime、descriptor 和 memory 五个源文件，公共 API 不依赖特定操作系统。平台通过回调提供 64-bit MMIO 读写、缓存维护和内存顺序保证；这些 MMIO 事务经系统 AXI 传输。

[!important] Conv2D 不是硬件原生指令 第一版没有 **CONV** opcode。对于 NHWC、**groups=1** 且 **stride**、**dilation**、**padding** 可由当前 DMA Descriptor 表达的 Conv2D，编译器形成 **padding / im2col / DMA FILL / COPY_ND / GEMM / bias / 输出整理** 的低层操作，并把对应的 CMD128、Descriptor、操作信息和命令分组写入模型专用 C 源码包。数据格式、分组、属性或所需工作区不符合要求时，在编译阶段报告错误，不生成 CPU im2col 函数。

主要部署文件是 **<stem>_model.c** 与 **<stem>_model.h**。低层 JSON IR、清单和反汇编文本可作为检查文件保留，但应用不需要在运行时解析模型文件，也不需要装载多个独立二进制镜像。

13. 第一版实现目标

1. 先完成 CMD128、CFE、TS、DFU、事件表和错误状态，使任务提交与依赖管理可验证。
2. 完成 DMA、L1BUF 和 MIF，再接入 Matrix 基本 GEMM，建立稳定的数据供给。
3. 加入 **MAC_CTX + ACCUM_CTX**，验证安全并行、部分和次序、最终事件时刻与性能计数。
4. 完成 Vector 和 Complex，覆盖 residual、门控、激活、Softmax 与 Norm。
5. 用 INT4、INT8、INT16 和 INT32 回归测试覆盖合法数据组合、溢出处理、不同 shape 和存储次序。
6. 使用同一套上层编译器、低层汇编器和 C 驱动生成可直接编译的模型专用 C 源码包，完成 Transformer、RNN、LSTM、GRU 与 **Conv2D(im2col)** 的端到端运行。
7. 验证外部 Generic Core 的 AXI Master 访问、NPU AXI Slave 命令提交、控制寄存器访问、L1BUF 外部窗口访问，以及 MIF 经 TBU 地址转换后从 **m_axi_*** 发出的系统总线 AXI 请求。

第一版优先证明单核功能、指令语义、软件生成结果与硬件执行结果一致；后续再依据性能计数调整 Matrix tile、L1BUF bank 数、Complex lane 数和 DMA 未完成请求数。